

基于 OpenC910 IEW 规格修改 gem5 O3 IEW 阶段 — 修改计划

一、架构差异总览

维度	OpenC910	gem5 O3 当前实现	差距
Issue Queue	8 个专用 IQ（共 86 条目）	统一 IQ（IQUnit 向量），按 OpClass 优先级队列	缺少指令类型路由/专用队列
执行 Pipe	8 条固定 Pipe，延迟明确	FU Pool 抽象，无固定 pipe 映射	缺少 pipe 级延迟模型
PRF	3 个独立 PRF（PREG/VREG/EREG）	统一 Scoreboard（bool 向量）	缺少多 PRF 分离管理
写回端口	3 PREG + 3 VREG + 2 EREG	单一 <code>wbWidth</code> 参数	缺少分类写回端口
Issue 宽度	8（跨 Queue 发射）	8（单一 IQ 宽度）	已有宽度，缺跨队列仲裁
LQ/SQ	16 LQ + 12 SQ 独立管理	LSQ 内部 <code>LoadQueue/StoreQueue</code>	容量一致，结构基本对齐
依赖检查	寄存器链表 + PST 记分板	DependencyGraph（链表/寄存器）	核心机制一致
年龄向量	显式 <code>agevec[i][j]</code> 矩阵	ReadyInstQueue 用 <code>seqNum</code> 排序	机制不同，效果类似
停顿类型	9 种（ROB/IQ/VMB/Type/...）	2-3 种（ROB/IQ/LSQ full）	缺少细粒度停顿
刷新来源	4 种（FE/IS/全局/CSR）	squash 单一信号	缺少分级刷新
前递路径	EX1 组合逻辑 + EX2 时钟沿	统一通过 <code>issueToExecQueue</code>	缺少分阶段前递
性能计数	10+ 类 HPCP 事件	~15 个统计项	需要补充事件

二、修改计划（按优先级排序）

Phase 1：多 Issue Queue 架构改造（最高优先级）

目标：将 gem5 统一 IQ 改造为 C910 式的 8 个专用 Issue Queue，支持按指令类型路由。

涉及文件：

文件	修改内容
<code>inst_queue.hh / inst_queue.cc</code>	核心改造：新增 IQType 枚举、路由逻辑、Entry 结构
<code>IQUnit.py</code>	新增 8 个专用 IQ 参数
<code>FUPool.py / FuncUnitConfig.py</code>	重命名/调整 FU 配置以匹配 8 Pipe

文件	修改内容
Base03CPU.py	新增 IQ 类型容量参数
iew.cc / iew.hh	dispatchInsts() 路由逻辑重写

1.1 定义 8 个 IQ 类型枚举 (inst_queue.hh)

```
enum class IQType {
    AIQ0, AIQ1,    // 整数 ALU (各 11 条目)
    BIQ,           // 分支/跳转 (12 条目)
    LSIQ,          // 加载指令 (16 条目)
    SDIQ,          // 存储数据 (8 条目)
    VIQ0, VIQ1,    // 向量计算 (各 10 条目)
    VMB,           // 向量内存缓冲 (8 条目)
    NUM_IQ_TYPES
};
```

1.2 指令类型 → IQ 路由表 (iew.cc 的 dispatchInsts())

当前路由逻辑简单区分 Load/Store/Atomic/Barrier/Nop/Other，需改为：

指令类型	目标 IQ	目的 Pipe	特殊条件
整数 ALU	AIQ0 或 AIQ1 (轮转/负载均衡)	Pipe0/Pipe1	可跨 Queue 灵活分配
乘加 (MUL/MADD)	AIQ1	Pipe1	Pipe1 独有 MLA 单元
除法 (DIV/REM)	AIQ0/AIQ1	Pipe0/Pipe1	独立除法单元，多周期
分支/跳转	BIQ	Pipe2	包含 CALL/RTS 标记
加载	LSIQ	Pipe3	需检查 LQ 非满
存储地址计算	LSIQ	Pipe3	地址计算与数据写入分离
存储数据写入	SDIQ	Pipe5	需检查 SQ 非满
FENCE/屏障	LSIQ	Pipe4	等待所有更早指令完成
向量 ALU	VIQ0/VIQ1	Pipe6/Pipe7	绑定对应 VFPU
向量乘累加	VIQ1	Pipe7	Pipe7 独有 VFMAU
向量内存	VMB	Pipe3/Pipe4/Pipe5	缓冲多个标量操作

1.3 IQ 容量参数 (Base03CPU.py)

```
aiq0Entries = Param.Unsigned(11, "AIQ0 entries")
aiq1Entries = Param.Unsigned(11, "AIQ1 entries")
biqEntries  = Param.Unsigned(12, "BIQ entries")
lsiqEntries = Param.Unsigned(16, "LSIQ entries")
sdiqEntries = Param.Unsigned(8,  "SDIQ entries")
viq0Entries = Param.Unsigned(10, "VIQ0 entries")
viq1Entries = Param.Unsigned(10, "VIQ1 entries")
vmbEntries  = Param.Unsigned(8,  "VMB entries")
```

1.4 IQ Entry 结构对齐 (inst_queue.hh)

C910 Entry 包含 vld, frz, src0_vld, src1_vld, agevec, opcode, iid, pid, pcall, rts。

gem5 已有部分状态 (isSquashed, isIssued, isExecuted) ，需补充：

```
bool frozen;           // frz - 等待同步条件 ( FENCE )
bool src0Ready;        // src0_vld
bool src1Ready;        // src1_vld
std::vector<bool> agevec; // 年龄向量
bool procedureCall;    // pcall
bool returnInst;       // rts
```

就绪条件对齐 C910: rdy = vld && src0_vld && src1_vld && !frz && ext_rdy

Phase 2：多 PRF + 分类写回端口

目标：将单一 Scoreboard + wbWidth 改造为 3 个独立 PRF + 分类写回端口。

涉及文件：

文件	修改内容
scoreboard.hh / scoreboard.cc	改造为多 PRF Scoreboard
iew.hh / iew.cc	writebackInsts() 多端口改造
BaseO3CPU.py	新增多 PRF 参数和多 wbWidth 参数
comm.hh	IEWStruct 可能需要扩展

2.1 多 PRF Scoreboard (scoreboard.hh)

```
class Scoreboard {
    // 替代单一 regScoreBoard
    std::vector<bool> pregScoreBoard; // 95 整数寄存器
    std::vector<bool> vregScoreBoard; // 64 向量寄存器
    std::vector<bool> eregScoreBoard; // 32 标量FP寄存器
    // 写回端口数
```

```
    unsigned pregWBPorts = 3;
    unsigned vregWBPorts = 3;
    unsigned eregWBPorts = 2;
};
```

2.2 分类写回端口参数 (Base03CPU.py)

```
pregWBWidth = Param.Unsigned(3, "Integer PRF writeback ports")
vregWBWidth = Param.Unsigned(3, "Vector PRF writeback ports")
eregWBWidth = Param.Unsigned(2, "Scalar FP writeback ports")
numPhysIntRegs    = Param.Unsigned(95, "Number of physical integer registers")
numPhysVecRegs    = Param.Unsigned(64, "Number of physical vector registers")
numPhysFloatRegs = Param.Unsigned(32, "Number of physical scalar FP registers")
```

2.3 写回端口映射

写回端口	来源	说明
PREG 端口 0	IU ALU Pipe0	整数运算结果
PREG 端口 1	IU ALU Pipe1	整数运算+乘加结果
PREG 端口 2	LSU Load Pipe3	加载数据
VREG 端口 0	VFPU Pipe6	向量运算结果
VREG 端口 1	VFPU Pipe7	向量运算结果
VREG 端口 2	LSU Load Pipe3	向量加载数据
EREG 端口 0	VFPU Pipe6	标量浮点结果
EREG 端口 1	VFPU Pipe7	标量浮点结果

2.4 写回逻辑改造 (iew.cc 的 writebackInsts())

当前逻辑：遍历 wbWidth 个槽位，统一写回。

改造后：按寄存器类型分类，各自独立端口计数：

```
对每个写回端口（并行执行）：
    若 wb_vld = 1:
        获取写回寄存器号 (wb_preg)
        更新对应 PRF Scoreboard (PREG/VREG/EREG)
        广播到依赖检查逻辑
        对所有 IQ 条目：
            若 src0_preg == wb_preg → src0_vld = 1
```

若 `src1_preg == wb_preg` $\rightarrow$ `src1_vld = 1`
前递数据到 IQ (组合逻辑路径)
发送完成信号到 RTU

Phase 3：8 条 Pipe + 执行延迟模型

目标：将 gem5 抽象 FU Pool 映射为 C910 的 8 条固定 Pipe，明确每条 Pipe 的延迟和流水线级数。

涉及文件：

文件	修改内容
<code>FuncUnitConfig.py</code>	重新定义 FU 延迟以匹配 8 Pipe
<code>FUPool.py</code>	重排 FU 列表映射 8 Pipe
<code>fu_pool.hh / fu_pool.cc</code>	支持 Pipe 编号映射
<code>inst_queue.cc</code>	<code>scheduleReadyInsts()</code> 适配 Pipe 分配

3.1 Pipe 延迟映射表 (`FuncUnitConfig.py`)

Pipe	单元	操作类型	延迟	pipelined
Pipe0	IU ALU	ADD/SUB/AND/OR/XOR/SLL/SRL/SRA/SLT	1 周期	true
Pipe1	IU ALU	同 Pipe0	1 周期	true
Pipe1	IU MLA	MUL/MULH/MULHSU/MULHU/MADD	3 周期	true
Pipe0/1	IU DIV	DIV/DIVU/REM/REMU	可变 (迭代)	false
Pipe2	BJU	BR/JAL/JALR/AUIPC	2 周期	true
Pipe3	LSU Load	LB/LH/LW/LD/LBU/LHU/LWU	可变	true
Pipe4	LSU Special	FENCE/FENCE.I/CSRR*	可变	true
Pipe5	LSU Store	SB/SH/SW/SD	1-2 周期	true
Pipe6	VFPU ALU	向量 ALU 操作	3-5 周期	true
Pipe7	VFPU MA	向量乘累加	3-5 周期	true

Phase 4：停顿/阻塞机制细化

目标：将 2-3 种停顿扩展为 C910 的 9 种停顿类型。

涉及文件：

文件	修改内容
<code>iew.hh</code>	新增停顿类型枚举

文件	修改内容
iew.cc	checkStall() 细化为多停顿检查
inst_queue.cc	新增 VMB 满检查

4.1 停顿类型定义 (iew.hh)

停顿类型	信号	原因	影响范围
ROB_FULL	ctrl_is_rob_full	退休队列满（64 条目用完）	阻止新指令创建到任何 IQ
IQ_FULL	ctrl_is_iq_full	某类型 Issue Queue 满	阻止该类型指令创建
VMB_FULL	ctrl_is_vmb_full	向量存储缓冲满	阻止向量内存指令创建
TYPE_STALL	is_dis_type_stall	屏障/FENCE 阻塞后续指令	阻止特定类型指令发射
DISPATCH_STALL	ctrl_is_dis_stall	Dispatch 停顿 (ROB/IQ/VMB Full)	阻止新指令创建
IS_STALL	ctrl_is_stall	IS 级总停顿	整个 IS 级暂停
DIV_STALL	iu_idu_div_wb_stall	除法单元写回阻塞	阻止除法指令发射
VDIV_STALL	vfp_u_idu_vdiv_wb_stall	向量除法写回阻塞	阻止向量除法发射
BACKEND_STALL	idu_hpcp_backend_stall	执行单元导致停顿	性能计数事件

4.2 停顿生成逻辑 (对齐 C910 Verilog)

```
// ctrl_is_dis_stall = ctrl_is_rob_full || ctrl_is_iq_full ||  
ctrl_is_vmb_full;  
// ctrl_is_stall = ctrl_is_dis_stall || is_dis_type_stall;  
bool checkDispatchStall(ThreadID tid) {  
    return isROBFull(tid) || isTargetIQFull(tid) || isVMBFull(tid);  
}  
bool checkISStall(ThreadID tid) {  
    return checkDispatchStall(tid) || isTypeStall(tid);  
}
```

Phase 5：分级刷新机制

目标：将单一 squash 信号改造为 4 种刷新来源，支持前端/后端/全局分级刷新。

涉及文件：

文件	修改内容
iew.hh	新增 FlushType 枚举
iew.cc	squash() 分级改造，新增 handleFlush()
comm.hh	CommitComm 新增刷新类型字段

5.1 刷新类型

来源	触发条件	刷新信号	效果
分支预测错误	BJU 检测到预测不匹配	flush_fe	清空前端（IR/IS/RF）
内存序违规	LSU 检测加载-存储顺序违规	flush_is	清空后端（IS/RF），保持前端
异常/陷阱	指令执行触发异常	flush_global	清空整个流水线
CSR 写特权寄存器	修改特权态或页表	flush_csr	清空前端，同步状态

5.2 刷新响应流程

接收刷新信号

├─ 若 flush_fe（前端刷新）：

├─ 清空 IS 级所有 IQ Entry (vld = 0)

├─ 重置年龄向量

├─ 清空 PCFIFO

└─ 重置 IQ 满标记

├─ 若 flush_is（后端刷新）：

├─ 清空 IS 级所有 IQ Entry

├─ 清空 RF 级暂存指令

└─ 保持 IR 级指令（前端不受影响）

└─ 若全局刷新：

执行前端刷新 + 后端刷新

5.3 取消发射（Cancel）信号

iu_yy_xx_cancel 信号：

├─ 来源：执行单元发现指令无法继续（如除法异常）

├─ 效果：取消当前正在发射的指令（Issue En 被阻止）

└─ 与刷新的区别：Cancel 仅阻止一条指令的发射

目标：区分 EX1 组合逻辑前递和 EX2 时钟沿写回，实现 C910 式的前递优先级。

涉及文件：

文件	修改内容
comm.hh	IssueStruct 新增前递字段
inst_queue.cc	前递比较逻辑，EX1/EX2 分离
fu_pool.hh	EX1/EX2 阶段分离

6.1 前递信号定义（对齐 C910）

来源	目标	阶段	信号
IU Pipe0 EX1	AIQ/AIQ1 条目	组合逻辑	iu_idu_ex1_pipe0_fwd_preg_data
IU Pipe1 EX1	AIQ/AIQ1 条目	组合逻辑	iu_idu_ex1_pipe1_fwd_preg_data
IU Pipe0 EX2 WB	AIQ/AIQ1 条目	时钟沿	iu_idu_ex2_pipe0_wb_preg_data
IU Pipe1 EX2 WB	AIQ/AIQ1 条目	时钟沿	iu_idu_ex2_pipe1_wb_preg_data
LSU Pipe3 EX1	LSIQ/AIQ 条目	组合逻辑	lsu_idu_da_pipe3_fwd_preg_data
LSU Pipe3 WB	LSIQ/AIQ 条目	时钟沿	lsu_idu_wb_pipe3_wb_preg_data
VFPU Pipe6 EX1	VIQ/AIQ 条目	组合逻辑	vfpu_idu_ex1_pipe6_mfvr_data
VFPU Pipe7 EX1	VIQ/AIQ 条目	组合逻辑	vfpu_idu_ex1_pipe7_mfvr_data

6.2 前递 vs 写回优先级

场景	机制	延迟
同一周期发射，源寄存器正在 EX1 前递路径	组合逻辑前递（FWD）	0 周期（同周期）
源寄存器在 EX2 写回阶段	时钟沿更新 + 下周期使用	1 周期
源寄存器在更早的 Pipe 中已完成	Scoreboard 标记就绪，RF 读命中	0 周期

6.3 前递比较逻辑

```
// 检查写回/前递寄存器是否匹配条目的源寄存器
assign src0_fwd = (wb_preg_dupx == src0_preg) && wb_preg_vld_dupx;
assign src1_fwd = (wb_preg_dupx == src1_preg) && wb_preg_vld_dupx;
// 前递命中 → 设置 src0_vld / src1_vld = 1
```

Phase 7：性能计数补充

目标：补充 C910 HPCP 事件到 gem5 统计系统。

涉及文件：

文件	修改内容
iew.hh	IEWStats 新增统计项
iew.cc	tick() 中采集事件
inst_queue.hh	IQStats 新增统计项

7.1 新增统计项

事件	信号	gem5 Stat
各 Pipe 发射计数	idu_hpcp_rf_pipe[0:7]_inst_vld	pipeIssueCount[8]
发射锁存失败	idu_hpcp_rf_pipe[0:7]_lch_fail_vld	issueLatchFail[8]
寄存器锁存失败	idu_hpcp_rf_pipe[3:5]_reg_lch_fail_vld	regLatchFail[3]
后端停顿	idu_hpcp_backend_stall	backendStallCycles
FENCE 同步	idu_hpcp_fence_sync_vld	fenceSyncCount
IQ 空	idu_had_iq_empty	iqEmptyCycles
Pipeline 空	idu_had_pipeline_empty	pipelineEmptyCycles
Pipeline 停顿	idu_had_pipe_stall	pipelineStallCycles

7.2 LSU 统计

事件	描述
D-Cache 读访问/缺失	加载指令的缓存行为
D-Cache 写访问/缺失	存储指令的缓存行为
加载/存储停顿	LSU 导致的停顿周期
未对齐访问	非对齐内存访问次数

三、文件修改清单

文件	修改量级	主要修改内容
iew.hh	大改	新增 IQType/停顿/刷新枚举、前递信号、多 wbWidth 成员
iew.cc	大改	dispatchInsts() 路由重写、checkStall() 细化、writebackInsts() 多端口、tick() 顺序调整
inst_queue.hh	大改	IQType 枚举、Entry 结构扩充、多 IQ 管理器、前递接口
inst_queue.cc	大改	scheduleReadyInsts() 跨 IQ 仲裁、前递逻辑、依赖唤醒、年龄向量

文件	修改量级	主要修改内容
scoreboard.hh	中改	分离为 PREG/VREG/EREG 三个 Scoreboard
scoreboard.cc	中改	多 PRF 初始化/操作
comm.hh	小改	CommitComm/IEWStruct 新增 FlushType 字段
Base03CPU.py	中改	新增 IQ 容量、多 PRF 参数、多 wbWidth 参数
FUPool.py	中改	重排 FU 列表映射 8 Pipe
FuncUnitConfig.py	中改	调整延迟对齐 C910 Pipe 延迟表
IQUnit.py	小改	支持 IQ 类型参数化
lsq.hh / lsq_unit.hh	小改	LQ/SQ 容量参数确认（已对齐 16/12）

四、实施顺序建议



- **Phase 1-3** 为架构级改造，改动量最大且互有依赖，必须按顺序依次完成
- **Phase 4-7** 为功能细化，Phase 4/5 可并行于 Phase 3 之后，Phase 6/7 可并行推进
- 每完成一个 Phase 后应运行 regression test 验证正确性